AIMS DTU Research Intern 2026

Agentic Deep Research

Preview

Build an agentic deep-research system over a corpus of recent LLM-agent papers, then measure how much each architectural component actually contributes to answer quality.

Large language models can answer most well-formed questions in a single forward pass, but they struggle with research-style questions that need evidence assembled from multiple sources, with citations, under uncertainty. The 2024-2026 wave of "deep research agents" attack this problem by running the LLM in a loop instead of as a one-shot function: the model decomposes the question, calls retrieval tools, reads what comes back, decides whether it has enough, and either searches again or writes a final answer with citations. A useful primer on this design space is the Deep Research Agents survey; the foundational ideas underneath it come from work like ReAct, Self-RAG, and Reflexion.

The agentic part is what makes these systems interesting and what makes them hard to evaluate. A fixed retrieval pipeline always behaves the same way, but an agent decides at runtime, based on what it has retrieved so far, whether to search again, refine its query, or stop and answer. That dynamism is also what makes them brittle: planning, retrieval quality, reflection-loop termination, and citation grounding can each silently degrade the final answer, and from the outside it all looks like "the LLM got it wrong." The only honest way to know which component is carrying the system is to ablate the components one at a time and measure what happens. A reasonable mental model for how to score the answers themselves is the LLM-as-judge paradigm; for retrieval-grounded evaluation, RAGAS is a standard reference.

Your Assignment

You are going to build one of these systems end-to-end on a fixed corpus, in a way that lets you cleanly turn each component off, and then write down what you found. The corpus is arXiv papers in cs.CL / cs.AI / cs.LG from Jan 2024 through Apr 2026 whose title or abstract concerns LLM agents, agentic RAG, tool use, agent memory, agent benchmarks, computer-use agents, or related topics, roughly 400-700 papers, collected via the free arXiv API. The exact filtering rules are your call.

The task has three broad phases:

• Collect the corpus from arXiv, parse the PDFs, chunk the text in whatever way you think preserves the signal, and build a retrieval index over it. The retrieval stack is your design problem: choose your embedding model, choose whether and how to combine lexical and semantic retrieval, choose your reranker if you have one, choose your vector store. Document your choices and why.

• Build the agent. The LLM must decide actions in a loop, not run a fixed pipeline. The system needs a way to plan (decompose the user's question into sub-questions), to retrieve and read (pull relevant passages, not just hand all chunks downstream), to reflect (decide whether the evidence is sufficient or whether to keep searching, capped at a sensible number of rounds), to synthesize (write an answer using only retrieved evidence and emit inline citations to specific arXiv IDs), and to verify citations (check that each claim is actually supported by its cited passage). The framework and the LLM backend are your call, as long as the entire stack runs on free tiers with no credit card on file anywhere.

• Evaluate. We provide a fixed evaluation set of 30 questions across factoid, comparative (≥2 papers), and survey (≥4 papers) types in eval/questions.jsonl, with the submission format specified in eval/SUBMISSION_FORMAT.md. We hold out the ground-truth answers and the "must-cite" arXiv ID sets and grade your submission against them. Run the system, a non-agentic baseline (single-shot retrieval plus one LLM call), and one ablation per component (no planner, no reranker if you have one, no reflector, no hybrid retrieval, no citation verifier). Produce one predictions/<config>.jsonl per configuration. In your report, score yourself with whatever judge you like (LLM-as-judge for answer accuracy and faithfulness, exact-set overlap for citation precision and recall) and report answer accuracy, faithfulness, citation precision, citation recall, latency, and tool-call count for each configuration. Produce one ablation table and write down what it tells you. Your numbers must match ours within tolerance when we re-grade.

Evaluation

The core question is whether the agentic scaffolding actually helps, and which parts of it carry the weight. It is entirely possible that on this corpus the agent is no better than a strong baseline, or that the reflector adds latency without improving accuracy, or that whatever you chose for reranking matters more than the planner.

Candidates are expected to make their own design choices, interpret results honestly, and be upfront about where their system fails and where their conclusions might be wrong. A submission that shows the agent winning everywhere with no caveats is less

interesting than one that shows the agent losing on factoid questions, explains why, and proposes what would fix it.

Deliverables

1. A technical report (4-6 pages) covering corpus construction, the architecture you converged on and the papers or sources that motivated each component, the evaluation and ablation tables, an honest interpretation of which components mattered and why, failure modes with a handful of worked examples, and directions for future work.

2. A GitHub repository with clean, reproducible code for the scraper, the index, the agent, and the evaluation pipeline. A single command should reproduce the numbers in the report from a fresh clone. The eval/questions.jsonl we provide, along with one predictions/<config>.jsonl per configuration (full agent, baseline, and each ablation), must live in the repo. A small browser demo with a "show trace" view (plan, retrievals, reflector decisions, final synthesis) is appreciated but not required; if included, link it from the report.